

DATA SCIENCE PROJECT GUIDE

Crop Diversity & Risk Exposure

across U.S. Farm Resource Regions

Python · GeoPandas · Folium | RMA SOB data, 1989--present | 9 crops, ~3,100 counties | Est. 4--6 weeks

CONTENTS

01 Environment setup	05 Diversity indices
02 Data ingestion	06 Risk pricing features
03 Spatial join	07 Analysis & modeling
04 Panel construction	08 Interactive map

PHASE 1 Environment & Tool Setup

~1 day

Create your Python environment

Use a dedicated virtual environment to keep dependencies clean. This project needs geospatial libraries that can conflict with other packages if installed globally.

```
# Create and activate environment
python -m venv crop-risk-env
source crop-risk-env/bin/activate # Windows: crop-risk-env\Scripts\activate

# Install core stack
pip install pandas numpy scipy scikit-learn
pip install geopandas fiona shapely pyproj
pip install folium branca
pip install matplotlib seaborn plotly
pip install jupyter notebook
```

Recommended project folder structure

```
crop-risk/
├── data/
│   ├── raw/           # original files, never modified
│   ├── processed/     # cleaned outputs from each step
│   └── shapefiles/    # county + FRR boundary files
└── notebooks/        # one notebook per phase
```

```
outputs/  
  figures/  
  maps/  
README.md
```

Get the Farm Resource Region shapefile

You will need two shapefiles: the USDA ERS Farm Resource Regions (9 regions), and the US Census TIGER county boundaries. Both are free downloads.

DOWNLOAD LINKS

USDA ERS Farm Resource Regions: ers.usda.gov/data-products/arms-farm-financial-and-crop-production-practices/

US Census TIGER counties: census.gov/geographies/mapping-files/time-series/geo/tiger-line-file.html

Download the 2020 county shapefile (cb-2020-us-county-500k.zip)

PHASE 2 Data Ingestion & Cleaning

~3--4 days

Parse the pipe-delimited SOB files

The RMA Summary of Business files use | as a delimiter. Load them carefully — some fields contain commas, which is exactly why the pipe delimiter was chosen.

```
import pandas as pd  
  
# Coverage level SOB (insurance risk data)  
sob_cov = pd.read_csv(  
    'data/raw/sobscov_1989forward.txt',  
    sep='|',  
    dtype={'location_state_code': str,  
           'location_county_code': str},  
    low_memory=False  
)  
  
# Cause of loss SOB  
sob_col = pd.read_csv(  
    'data/raw/colsommonth_allyears.txt',  
    sep='|',  
    dtype={'state_code': str, 'county_code': str},  
    low_memory=False  
)  
  
# Build 5-digit FIPS code for joining  
sob_cov['fips'] = (sob_cov['location_state_code'].str.zfill(2)  
                  + sob_cov['location_county_code'].str.zfill(3))
```

WATCH OUT

State and county codes must be zero-padded as strings, not integers. Loading them as numeric will drop leading zeros — Montana state code 30 is fine, but county code 003 becomes 3 and won't join correctly. Always use `dtype=str` for FIPS columns.

Load and clean the cost & return CSVs

Each of the 9 crop CSVs has a similar structure. Build a unified long-format DataFrame by loading and concatenating them.

```

crops = ['Corn', 'Soybean', 'Wheat', 'Cotton',
         'Sorghum', 'Rice', 'Oats', 'Barley', 'Peanut']

frames = []
for crop in crops:
    df = pd.read_csv(f'data/raw/{crop}CostReturn.csv')
    df['crop'] = crop
    frames.append(df)

cost_return = pd.concat(frames, ignore_index=True)
cost_return.to_parquet('data/processed/cost_return_panel.parquet')

```

Compute loss ratios and filter bad rows

Check for rows where liability or premium is zero. These represent policies with no financial exposure and should be excluded from loss ratio calculations to avoid division-by-zero errors.

```

# Drop rows with zero premium (can't compute loss ratio)
sob_cov = sob_cov[sob_cov['total_premium_amount'] > 0]

# Compute loss ratio
sob_cov['loss_ratio'] = (sob_cov['indemnity_amount']
                        / sob_cov['total_premium_amount'])

# Winsorize extreme values (loss ratios > 10 are data artifacts)
sob_cov['loss_ratio'] = sob_cov['loss_ratio'].clip(upper=10)

```

PHASE 3 Spatial Join — County to FRR

~1--2 days

Load shapefiles and perform the join

Each county needs to be assigned to one of the 9 USDA Farm Resource Regions. Spatially join county centroids to FRR polygons. The result is a simple lookup table: FIPS → FRR name.

```

import geopandas as gpd

counties = gpd.read_file('data/shapefiles/cb_2020_us_county_500k.shp')
frr = gpd.read_file('data/shapefiles/frr_boundaries.shp')

```

```
# Ensure same coordinate reference system
counties = counties.to_crs(frr.crs)

# Build FIPS code in the county shapefile
counties['fips'] = counties['STATEFP'] + counties['COUNTYFP']

# Spatial join using county centroids
counties['centroid'] = counties.geometry.centroid
county_pts = counties.set_geometry('centroid')
joined = gpd.sjoin(county_pts, frr[['frr_name', 'geometry']],
                  how='left', predicate='within')

# Save the FIPS -> FRR lookup
fips_to_frr = joined[['fips', 'frr_name']].drop_duplicates()
fips_to_frr.to_csv('data/processed/fips_frr_lookup.csv', index=False)
```

TIP

A handful of coastal or border counties may not fall cleanly inside an FRR polygon. Use a nearest-neighbor fallback for unmatched counties: `gpd.sjoin_nearest()` will assign them to the closest region rather than leaving them null.

PHASE 4 Build the Analysis Panel

~2 days

Aggregate to county × year level

For each county and year, compute a liability-weighted average loss ratio across all crops. This gives larger, more financially significant policies more influence on the county's overall risk score.

```
# Weighted average loss ratio per county-year
county_year = (sob_cov
               .groupby(['fips', 'commodity_year'])
               .apply(lambda x: pd.Series({
                   'loss_ratio_wt': (x['loss_ratio'] * x['liability_amount']).
                       sum()
                                   / x['liability_amount'].sum(),
                   'total_liability': x['liability_amount'].sum(),
                   'total_premium': x['total_premium_amount'].sum(),
                   'total_indemnity': x['indemnity_amount'].sum(),
                   'n_crops_insured': x['commodity_code'].nunique()
               })))
               .reset_index()
               )

# Merge in FRR assignment
county_year = county_year.merge(fips_to_frr, on='fips', how='left')
```

Add crop acreage breakdown per county-year

Extract net reported quantity (acres) per crop per county per year from the SOB coverage file. This is the input for the diversity indices in the next phase.

```
# Acres per crop per county-year
acreage = (sob_cov[sob_cov['quantity_type'].str.lower() == 'acres']
           .groupby(['fips', 'commodity_year', 'commodity_name'])['
               net_reported_quantity']
           .sum()
           .reset_index()
)

# Pivot to wide format: one row per county-year, one col per crop
acreage_wide = acreage.pivot_table(
    index=['fips', 'commodity_year'],
    columns='commodity_name',
    values='net_reported_quantity',
    fill_value=0
).reset_index()
```

PHASE 5 Compute Diversity Indices

~2 days

Shannon entropy (primary diversity measure)

Shannon's H measures both the number of crops present and how evenly acreage is distributed across them. A county with 5 crops at 20% each scores higher than one where a single crop dominates at 80%.

$$H = - \sum_{i=1}^n p_i \ln(p_i)$$

where p_i is the proportion of total insured acreage devoted to crop i .

```
import numpy as np

def shannon_entropy(row):
    vals = row[row > 0]
    if len(vals) == 0: return 0
    props = vals / vals.sum()
    return -(props * np.log(props)).sum()

crop_cols = [c for c in acreage_wide.columns
              if c not in ['fips', 'commodity_year']]

acreage_wide['shannon_h'] = acreage_wide[crop_cols].apply(
    shannon_entropy, axis=1)
acreage_wide['n_crops'] = (acreage_wide[crop_cols] > 0).sum(axis=1)
```

```
# Herfindahl-Hirschman Index (concentration -- inverse of diversity
)
def hhi(row):
    vals = row[row > 0]
    if len(vals) == 0: return 1
    props = vals / vals.sum()
    return (props ** 2).sum()

acreage_wide['hhi'] = acreage_wide[crop_cols].apply(hhi, axis=1)
# HHI of 1.0 = total monoculture; ~0.1 = highly diversified
```

Price volatility from cost/return data

For each crop and FRR, compute the coefficient of variation (CV) of price over rolling 5-year windows. Merge back as an exposure-weighted average across crops present in each county.

```
# Rolling 5-year CV of price per crop per FRR
price_vol = (cost_return
.groupby(['frr_name', 'crop', 'year'])['price_per_unit']
.mean()
.reset_index()
.sort_values(['frr_name', 'crop', 'year'])
)

price_vol['price_cv_5yr'] = (price_vol
.groupby(['frr_name', 'crop'])['price_per_unit']
.transform(lambda x: x.rolling(5).std() / x.rolling(5).mean())
)
```

PHASE 6 Risk Pricing Features

~2 days

Build the cause-of-loss peril profile

From the cause-of-loss file, compute the share of total indemnity attributable to each peril type (drought, excess moisture, freeze, wind, etc.) for each county-year. This feature vector describes *what kind* of risk each county faces.

```
# Total indemnity by peril per county-year
peril_share = (sob_col
.groupby(['fips', 'commodity_year', 'cause_of_loss_description'])
['indemnity_amount']
.sum()
.reset_index()
)

# Convert to share of total indemnity
total_indem = peril_share.groupby(
    ['fips', 'commodity_year'])['indemnity_amount'].transform('sum'
)
peril_share['peril_share'] = (peril_share['indemnity_amount']
```

```

                                / total_indem.replace(0, np.nan))

# Pivot to wide: drought_share, moisture_share, freeze_share, etc.
peril_wide = peril_share.pivot_table(
    index=['fips', 'commodity_year'],
    columns='cause_of_loss_description',
    values='peril_share',
    fill_value=0
).reset_index()

```

Merge everything into the master panel

Join all features onto the county-year base table. This is your analytical dataset — one row per county per year with diversity indices, loss ratio, price volatility, peril profile, and FRR assignment.

```

panel = (county_year
    .merge(acreage_wide[['fips', 'commodity_year', 'shannon_h', 'hhi',
        'n_crops']],
        on=['fips', 'commodity_year'], how='left')
    .merge(peril_wide,
        on=['fips', 'commodity_year'], how='left')
)

panel.to_parquet('data/processed/master_panel.parquet')
print(f Panel shape: {panel.shape} )
# Expect ~60,000-90,000 rows (county-years with insurance data)

```

PHASE 7 Analysis & Modeling

~1 week

Exploratory: diversity vs. loss ratio

Start with scatter plots and correlation matrices, broken out by FRR. The relationship between diversity and risk may be strong in the Northern Crescent but weak in the Prairie Gateway where monoculture is structurally dominant. Look for this heterogeneity before fitting any global model.

Panel regression (primary model)

Use a fixed-effects panel regression to control for time-invariant county characteristics (soil quality, climate zone) and year fixed effects (national policy changes, macro weather events). The coefficient on Shannon H is the core estimate.

```

from linearmodels.panel import PanelOLS

panel = panel.set_index(['fips', 'commodity_year'])

model = PanelOLS(
    dependent = panel['loss_ratio_wt'],

```

```

exog      = panel[['shannon_h', 'hhi', 'drought_share', '
                  total_liability']],
entity_effects = True,      # county fixed effects
time_effects  = True       # year fixed effects
)
result = model.fit(cov_type='clustered', cluster_entity=True)
print(result.summary)

```

Random forest for nonlinear effects

The diversity–risk relationship may not be linear — there could be a threshold effect where diversity only reduces risk above a certain level. Train a random forest with Shannon H , HHI, peril shares, and FRR dummies as features. Use SHAP values to interpret which features drive predictions in each region.

KEY OUTPUTS FROM THIS PHASE

Coefficient table showing diversity’s effect on loss ratio by FRR. Feature importance plot from random forest. Scatter plots of diversity vs. loss ratio colored by drought exposure share. Rolling correlation of diversity and loss ratio over time to spot regime changes.

PHASE 8 Interactive Map Dashboard

~3--4 days

Build the choropleth with Folium

Folium wraps Leaflet.js and produces self-contained HTML maps. Build a county-level choropleth where color encodes Shannon diversity index, with a tooltip showing loss ratio, dominant crop, FRR name, and peril profile on hover.

```

import folium

# Merge panel summary onto county geometries
map_data = counties.merge(
    panel.groupby('fips').agg(
        shannon_h_mean=('shannon_h', 'mean'),
        loss_ratio_mean=('loss_ratio_wt', 'mean'),
    ).reset_index(),
    on='fips', how='left'
)

m = folium.Map(location=[38, -96], zoom_start=4,
               tiles='CartoDB positron')

folium.Choropleth(
    geo_data=map_data.to_json(),
    data=map_data,
    columns=['fips', 'shannon_h_mean'],
    key_on='feature.properties.fips',
    fill_color='YlGn',
    fill_opacity=0.7,

```



```

        line_opacity=0.2,
        legend_name='Crop diversity (Shannon H)',
        nan_fill_color='#f0f0f0'
    ).add_to(m)

# Add hover tooltips
folium.GeoJson(
    map_data,
    style_function=lambda x: {'fillOpacity': 0, 'weight': 0},
    tooltip=folium.GeoJsonTooltip(
        fields=['NAME', 'frr_name', 'shannon_h_mean', '
            loss_ratio_mean'],
        aliases=['County', 'FRR', 'Diversity (H)', 'Avg loss ratio'
        ]
    )
).add_to(m)

m.save('outputs/maps/diversity_risk_map.html')

```

Add layer toggles for multiple views

Use Folium's `LayerControl` to let users switch between viewing diversity, loss ratio, drought exposure share, and the residual (counties where the model predicts low risk but actual loss ratio is high — potential underpricing flags).

```

# Add each metric as its own choropleth layer
layers = {
    'Crop diversity (Shannon H)': ('shannon_h_mean', 'YlGn'),
    'Avg loss ratio': ('loss_ratio_mean', 'YlOrRd'),
    'Drought exposure share': ('drought_share', 'OrRd'),
    'Diversity-risk residual': ('model_residual', 'RdBu'),
}

for name, (col, palette) in layers.items():
    folium.Choropleth(
        ..., name=name,
        show=(name == 'Crop diversity (Shannon H)')
    ).add_to(m)

folium.LayerControl().add_to(m)
m.save('outputs/maps/diversity_risk_map_final.html')

```

Deliverables

PRIMARY DELIVERABLE

Interactive county-level map (HTML)

A self-contained Folium/Leaflet map with layer toggles for diversity, loss ratio, drought share, and model residuals. Hover tooltips show county name, FRR, dominant crop, and key statistics. Filterable by year range. Shareable as a single HTML file — no server required.

STATISTICAL OUTPUT

Panel regression results

Coefficient table showing diversity's effect on loss ratio by FRR, with confidence intervals and fixed-effect controls.

VISUAL

Diversity index time series

Line charts showing how Shannon H has changed per FRR since 1989, annotated with major Farm Bill policy events.

VISUAL

Peril profile heatmap

FRR $\times$ peril matrix showing what share of losses come from drought, flood, freeze, and wind in each region.

PRACTICAL OUTPUT

Diversity risk scorecard

Ranked table of counties by diversity score, loss ratio, and the gap between predicted and actual risk.

Tech Stack Reference

Library	Purpose
pandas	Data wrangling, panel construction, aggregations
numpy / scipy	Diversity index math, statistical tests
geopandas	Shapefile loading, spatial joins, county geometries
linearmodels	Fixed-effects panel regression (PanelOLS)
scikit-learn	Random forest, feature importance
shap	Model interpretability — which features drive predictions
folium	Interactive choropleth maps (Leaflet.js wrapper)
matplotlib / seaborn	Static charts, correlation matrices, time series
plotly	Interactive scatter plots, FRR comparison charts
pyarrow / parquet	Efficient storage of large intermediate DataFrames

Suggested Timeline

Week	Focus
Week 1	Environment setup, data ingestion, and cleaning
Week 2	Spatial join and panel construction
Week 3	Diversity indices and price volatility features
Week 4	EDA, panel regression, and random forest
Week 5	Interactive map and layer controls
Week 6	Diversity risk scorecard, methodology notebook, and writeup